

THÈSE DE DOCTORAT

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Davide FISSORE

Centre Inria d'Université Côte d'Azur, équipe STAMP

**Présentée en vue de l'obtention
du grade de docteur en Informatique
d'Université Côte d'Azur**

Dirigée par : Yves BERTOT, Directeur de
recherche, Centre Inria d'Université Côte
d'Azur

Soutenue le : 16 septembre 2026

Devant le jury, composé de :

Enrico TASSI, Chargé de recherche, Centre Inria d'Université Côte d'Azur
Catherine DUBOIS, Professeur, ENSIIE
Matthieu SOZEAU, Chargé de recherche, Université de Nantes
Chantal KELLER, Maîtresse de Conférences, Université Paris-Saclay
Assia MAHBOUBI, Directrice de recherche, Université de Nantes

Programmation logique pour les preuves interactives :
inférence des classes de type et analyse du déterminisme

*Logic programming for interactive proofs:
Type Classes inference and Determinacy Analysis.*

Davide Fissore



Jury :

Rapporteurs

Catherine DUBOIS, Professeur, ENSIIE

Matthieu SOZEAU, Chargé de recherche, Université de Nantes

Examineurs

Enrico TASSI, Chargé de recherche, Centre Inria d'Université Côte d'Azur

Chantal KELLER, Maîtresse de Conférences, Université Paris-Saclay

Assia MAHBOUBI, Directrice de recherche, Université de Nantes

Directeur de thèse

Yves BERTOT, Directeur de recherche, Centre Inria d'Université Côte d'Azur

Davide FISSORE

*Programmation logique pour les preuves interactives : inférence des
classes de type et analyse du déterminisme*

ix+8 p.

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Résumé

La résolution de classes de types est un mécanisme central de l'élaboration dans les assistants de preuve modernes, assurant l'inférence d'arguments implicites et la désambiguïsation des symboles surchargés. Sa proximité avec la programmation logique suggère des implémentations alternatives fondées sur l'exécution de programmes logiques.

Cette thèse propose une implémentation d'un solveur de classes de types pour Rocq basé sur ELPI, un interpréteur Lambda-Prolog emarqué. Nous introduisons une compilation des classes de types et instances vers une base de connaissances logique, permettant de déléguer la résolution à ELPI. Nous analysons l'expressivité de cette approche et ses performances en la comparant au solveur natif de Rocq.

Un défi majeur provient des divergences entre Rocq et ELPI en matière d'unification et de représentation des termes. Bien que les deux systèmes supportent l'unification d'ordre supérieur et la $\beta\eta$ -conversion, leurs encodages diffèrent, ce qui peut empêcher la préservation de l'unification après traduction. Pour remédier à ce problème, nous développons une technique de compilation fondée sur des contraintes différées : les sous-termes problématiques sont remplacés par des liens enregistrant des contraintes qui seront vérifiées au cours de l'exécution, ce qui permet de retrouver le comportement d'unification attendu.

Nous étudions également la prédictibilité des programmes logiques au moyen d'une analyse de déterminisme pour ELPI. Celle-ci garantit qu'un prédicat donné produit au plus une solution, en supportant prédicats d'ordre supérieur, clauses dynamiques et cut. Cette analyse est particulièrement pertinente dans le contexte de la résolution de classes de types, où un comportement déterministe est généralement attendu. Enfin, nous formalisons la correction de cette analyse dans Rocq via un modèle d'ordre un de l'interpréteur ELPI et une preuve de sûreté du vérificateur statique.

Dans l'ensemble, ce travail met en évidence le rôle des techniques de programmation logique dans la mise en œuvre et l'analyse de mécanismes d'élaboration, éclairant les liens entre résolution de classes de types, programmation logique et vérification formelle.

Mots-clés : Programmation logique, Théorie des types, Elpi, Rocq, Classes de type, Analyse de déterminisme.

Logic programming for interactive proofs: Type Classes inference and Determinacy Analysis.

Abstract

Type-class resolution is a central component of elaboration in modern proof assistants, providing implicit argument inference and the disambiguation of overloaded symbols. Its close relationship with logic programming suggests alternative implementations based on the execution of logic programs.

This thesis presents an implementation of a type-class solver for the Rocq proof assistant based on ELPI, an embeddable Lambda-Prolog interpreter. We introduce a compilation scheme that translates type classes and their instances into a logic-programming knowledge base, allowing type-class resolution to be delegated to ELPI. We study the expressiveness of this approach and evaluate its performance by comparing it with Rocq’s native solver.

A major challenge arises from the differences between Rocq and ELPI with respect to unification and term representation. Although both systems support higher-order unification and $\beta\eta$ -conversion, their term encodings differ, which may prevent unification from being preserved by the translation. To address this issue, we develop a compilation technique based on delayed constraints: problematic subterms are replaced with links recording constraints that are checked during execution, thereby recovering the intended unification behavior.

We also investigate the predictability of logic programs through a determinacy analysis for ELPI. The analysis guarantees that a given predicate produces at most one solution while supporting higher-order predicates, dynamic clauses, and the cut operator. This analysis is particularly relevant to type-class resolution, where deterministic behavior is generally expected. Finally, we formalize the correctness of this analysis in Rocq by mechanizing a first-order model of the ELPI interpreter and proving the soundness of the static checker.

Overall, this work demonstrates the role that logic-programming techniques can play in the implementation and analysis of elaboration mechanisms, shedding new light on the connections between type-class resolution, logic programming, and formal verification.

Keywords: Logic Programming, Type Theory, Elpi, Rocq, Type Classes, Determinacy Analysis.

Acknowledgement

TODO

Contents

1	Introduction	1
1.1	Contributions and structure of the thesis	4
	Bibliography	5

CHAPTER 1

Introduction

Logical reasoning plays a central role in the way humans justify actions, choices, and conclusions, particularly when decisions are not primarily driven by emotions or other non-rational factors.

For example, if “All men are mortal” and “Socrates is a man”, then we can conclude that “Socrates is mortal”. This particular form of reasoning, known as an Aristotelian syllogism, is irrefutable: the conclusion follows directly from the premises.

A counterpart to these philosophical ideas can be found in mathematical reasoning. Much like a syllogism, the validity of a mathematical statement is established through a proof constructed from a collection of axioms and inference rules.

In his 1879 work *Begriffsschrift* (“concept writing” in English), Gottlob Frege attempted to formalize a complete logical foundation for mathematics based on a set of axioms.

The axioms underlying a theory must satisfy a fundamental requirement: they should not give rise to paradoxes, otherwise the theory itself becomes meaningless. Frege’s logical system was later shown to be inconsistent following Russell’s discovery of the paradox that now bears his name. Today, several foundational frameworks coexist, including classical set theory, intuitionistic logic, and various forms of type theory.

In the middle of the twentieth century, with the advent of computer science, Curry and Howard [6] discovered a deep connection between the representation of propositions and their proofs on the one hand, and computation on the other. Under this correspondence, a proof of a proposition can be viewed as a well-typed program, while the proposition itself corresponds to the type of that program.

Using software to construct proofs has increasingly become a viable alternative to pen-and-paper proofs: once a proof is provided to a machine, it becomes the machine’s responsibility to verify that the proof guarantees the desired property. Proof assistants are software systems designed to support the construction and verification of formal proofs while providing interactive human-machine collaboration. As a result, one can state with a high degree of confidence that a program satisfies a formally verified specification.

For example, the operator $+$ can be described as a program that takes two integers and computes their sum. Within a proof assistant, it is possible to prove in a single line of code, using induction, that 0 is a neutral element for addition, i.e., $\forall a, a + 0 = a$.

The importance of certified algorithms is particularly evident in safety-critical domains such as medicine and aerospace engineering.

In such settings, the tolerance for implementation errors is extremely low. For instance, when launching a rocket into space, it is unacceptable for the mission to fail because of a

software bug. A well-known example is the maiden flight of the Ariane 5 rocket in 1996, which ended in failure due to a software error.

Beyond checking proofs, proof assistants also help users write proofs by automating parts of the proving process.

In general, formal proofs developed in a proof assistant must be extremely detailed and are often difficult for humans to read. To reduce the effort required during proof development, modern proof assistants, such as the ROCQ [1] proof assistant (formerly known as COQ), provide various forms of automation that allow users to write incomplete proof terms, leaving some information to be inferred automatically by the system.

These mechanisms are collectively referred to as the *elaborator* [11]. The purpose of elaboration is to bridge the gap between the notation written by users and the internal representation manipulated by the proof assistant. Just as a computer does not directly understand ordinary text, the elaborator translates user-written terms into a machine-interpretable form. Moreover, elaboration allows incomplete proof terms to be submitted and automatically completed by the system. In some situations, ambiguities may even be left intentionally in the source code, with the elaborator responsible for resolving them using information available in the current environment.

A common source of ambiguity arises from symbol overloading. In mathematics, it is customary to reuse the same notation in different contexts. For example, the symbol $+$ may denote addition over natural numbers in the expression $1 + 2$, or vector addition in the expression $[\mathbf{p}_i, e] + [\sqrt{2}, \frac{1}{3}]$. A human reader can usually infer the intended meaning from the surrounding context.

A similar ambiguity exists in programming languages. In object-oriented languages, a common interface may admit multiple implementations, and the compiler is responsible for selecting the appropriate one. Functional programming languages vary in the support they provide for overloading. For instance, in OCAML, different operators are typically required for different notions of addition, whereas HASKELL supports overloading through its type-class mechanism [12]. Since many proof assistants are based on functional programming languages, they often provide a type-class system as well. In ROCQ, for example, type classes were introduced in 2008 [9].

The role of the type-class solver is to determine which implementation should be used in a given context. Conceptually, the expression $1 +_? 2$ can be viewed as containing a missing piece of information, represented here by the placeholder $?$: the implementation of addition appropriate for the current types. The task of the type-class solver is to infer this implementation automatically.

As observed in [11], elaboration mechanisms are often based on inference rules reminiscent of logic programming. In this work, we investigate an alternative implementation of type-class resolution based on a logic programming language. This approach offers an immediate advantage: the possibility of reusing existing logic-programming techniques developed by the logic-programming community, as well as existing inference engines, rather than reimplementing the resolution procedure from scratch. At the same time, it raises several questions. How naturally can ROCQ’s inference problems be translated into a logic-programming setting? What are the performance implications of such a translation? More generally, what are the benefits and limitations of this design choice?

To explore these questions, we rely on ELPI [2], the *Embeddable Lambda-Prolog Interpreter*. On the one hand, ELPI is an implementation of λ -PROLOG [8]; on the other hand,

it integrates easily with a host application such as ROCQ. The ROCQ-ELPI plugin [10] provides such an integration between ROCQ and ELPI.

This setup gives rise to a form of metaprogramming in which ROCQ serves as the object language and delegates part of its computation to ELPI, the meta-language. The key idea is that the object language is embedded within the meta-language, meaning that terms of the former are translated into terms of the latter. In our encoding, terms are represented using a shallow embedding, allowing users to inspect and manipulate them directly. Furthermore, a collection of APIs for the object language is exposed, enabling the meta-language to leverage some of the object language’s functionalities.

In this work, we study the integration of ROCQ with an alternative type-class solver implemented in ELPI. This requires translating type classes and instances from ROCQ into an ELPI database, which can be viewed as a compilation step. We show in ?? that a basic compiler can be written in only a few lines while relying on the existing ELPI interpreter for resolution. We then extend this translation to support more complex instances and compare the resulting solver with ROCQ’s native solver ??????.

A central challenge in this approach arises from the mismatch between the object language and the meta-language. Although both support higher-order unification [7] and $\beta\eta$ -conversion, the shallow encoding of terms, which underlies our metaprogramming setup, creates terms whose λ -abstractions and applications are represented using rigid constructors rather than the native constructs of the meta-language. As a consequence, additional constraints are introduced during unification.

As a result, terms that are unifiable in ROCQ may fail to unify after translation into ELPI. Simply delegating unification to the meta-language is therefore insufficient. This motivates another contribution of this thesis.

To address this unification mismatch, we propose a compilation scheme in ?? that preserves most of the structure of terms while isolating problematic subterms. These subterms are replaced with unification variables only when necessary and recorded to the original term with *links*, representing delayed constraints. These links are checked during execution, allowing us to recover the intended behavior without reimplementing unification.

Beyond type-class resolution, the intrinsic nature of logic programming raises another important concern, especially for users unfamiliar with logic programming. Controlling the backtracking performed by the interpreter is fundamental to managing performance. This motivates a second line of work developed in this thesis: a determinacy analysis for ELPI programs ?. Determinacy ensures that selected predicates produce at most one solution and do not introduce unexpected choice points for any call. This property helps control the behavior of logic programs. In our setting, type classes correspond to predicates and instances to rules. If a class is declared deterministic, the analysis ensures that the program generated by the instances of this class satisfies the property that any call to the class produces at most one solution.

We design and implement an analysis based on inspecting clauses and tracking argument modes. The algorithm supports most features of ELPI, including higher-order predicates, dynamic clauses, and the cut operator. Handling cut is important, as it can enforce determinism while interacting in subtle ways with logical semantics.

To increase confidence in the static analysis, we formalize the expected properties of the checker in the ROCQ proof assistant by encoding the ELPI interpreter ?. To simplify

this development, we restrict the model to a first-order fragment of the language, where terms do not contain higher-order variables. In this setting, the absence of higher-order variables simplifies the formalization and allows us to produce a fully mechanized proof.

Although this model does not capture all higher-order features of ELPI, it encompasses the aspects relevant to our analysis and provides a first foundation for establishing machine-checked correctness guarantees for the ELPI language.

1.1 Contributions and structure of the thesis

This thesis is organized as follows. We first present the context of the work (??). In particular, we describe the type-class resolution mechanism implemented in ROCQ, together with its main options. Then, we introduce the ELPI language and give a formal description of its syntax and semantics. Finally, we present the ROCQ-ELPI plugin, which enables the interaction between ELPI and ROCQ.

This thesis contains five main contributions, organized as follows:

???? We present a formalization of the ELPI language, covering both its syntax and its semantics. This formalization first appeared in the appendix of [5]. We include it in ??, the chapter describing the context of this work, because this placement fits the overall structure of the thesis better.

?? We study the implementation of a type-class solver written in ELPI for ROCQ [3]. We present the core part of the compiler that translates ROCQ classes and instances into an ELPI database. This database is then used by the interpreter to search for inhabitants of type classes. We also compare our solver with ROCQ’s native solver, focusing on performance, modes, and unification issues.

?? We analyze unification problems arising from the higher-order abstract syntax used to represent ROCQ terms in ELPI. In ROCQ, terms that are equal up to $\beta\eta$ -conversion can unify, while in ELPI such unification may fail, even if its algorithm supports these conversions. Based on our work [4], we explain how to address this issue by delaying problematic unifications using a system of links.

?? We present a static determinacy checker implemented in ELPI, available since version 3. This tool verifies that predicates declared as deterministic by the user effectively behave deterministically, meaning that they never create additional choice points. This result is based on our work [5].

?? We mechanize the determinacy checker in the ROCQ proof assistant. The formalization considers a restricted fragment of ELPI, namely its first-order part, where variables only represent data. Within this setting, we prove the main determinacy property: in a program that satisfies the checker, any call to a deterministic predicate produces at most one solution.

Bibliography

167

168

- 169 [1] Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program De-*
170 *velopment. Coq'Art: The Calculus of Inductive Constructions*. Texts in Theoretical
171 Computer Science. Springer Verlag, 2004.
- 172 [2] Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI:
173 fast, embeddable, λ Prolog interpreter. In *Proceedings of LPAR*, volume 9450 of
174 *LNCS*, pages 460–468. Springer, 2015. URL: [https://inria.hal.science/hal-](https://inria.hal.science/hal-01176856v1)
175 [01176856v1](https://inria.hal.science/hal-01176856v1), doi:10.1007/978-3-662-48899-7_32.
- 176 [3] Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The*
177 *Coq Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 178 [4] Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-
179 language unification for the object language. In *Proceedings of PPDP*, pages 1–13.
180 ACM, 2024. doi:10.1145/3678232.3678233.
- 181 [5] Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order
182 logic programming language with cut. In Nada Amin and Joaquín Arias, editors,
183 *Practical Aspects of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature
184 Switzerland.
- 185 [6] William Alvin Howard. The formulae-as-types notion of construction. In Haskell
186 Curry, Hindley B., Seldin J. Roger, and P. Jonathan, editors, *To H. B. Curry: Essays*
187 *on Combinatory Logic, Lambda Calculus, and Formalism*. Academic Press, 1980.
- 188 [7] Dale Miller. A logic programming language with lambda-abstraction, function vari-
189 ables, and simple unification. In *Extensions of Logic Programming*, pages 253–281.
190 Springer, 1991.
- 191 [8] Gopalan Nadathur and Dale A. Miller. An overview of lambda-prolog. In *Proceedings*
192 *of the Fifth International Conference and Symposium on Logic Programming*, pages
193 810–827, Seattle, WA, USA, 1988. MIT Press.
- 194 [9] Matthieu Sozeau and Nicolas Oury. First-class type classes. In *Proceedings of*
195 *TPHOLs*, volume 5170 of *LNCS*, pages 278–293. Springer, 2008.
- 196 [10] Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi
197 λ Prolog dialect). In *The Fourth International Workshop on Coq for Programming*
198 *Languages*, January 2018. URL: <https://inria.hal.science/hal-01637063>.
- 199 [11] Enrico Tassi and Freek Wiedijk. Todo. In TODO, editor, *TODO*, 2026.
- 200 [12] Philip Wadler and Stephen Blott. How to make ad-hoc polymorphism less ad hoc. In
201 *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Pro-*
202 *gramming Languages*, POPL '89, page 60–76, New York, NY, USA, 1989. Association
203 for Computing Machinery. doi:10.1145/75277.75283.

Programmation logique pour les preuves interactives : inférence des classes de type et analyse du déterminisme

Résumé

La résolution de classes de types est un mécanisme central de l'élaboration dans les assistants de preuve modernes, assurant l'inférence d'arguments implicites et la désambiguïsation des symboles surchargés. Sa proximité avec la programmation logique suggère des implémentations alternatives fondées sur l'exécution de programmes logiques.

Cette thèse propose une implémentation d'un solveur de classes de types pour Rocq basé sur ELPI, un interpréteur Lambda-Prolog emarqué. Nous introduisons une compilation des classes de types et instances vers une base de connaissances logique, permettant de déléguer la résolution à ELPI. Nous analysons l'expressivité de cette approche et ses performances en la comparant au solveur natif de Rocq.

Un défi majeur provient des divergences entre Rocq et ELPI en matière d'unification et de représentation des termes. Bien que les deux systèmes supportent l'unification d'ordre supérieur et la $\beta\eta$ -conversion, leurs encodages diffèrent, ce qui peut empêcher la préservation de l'unification après traduction. Pour remédier à ce problème, nous développons une technique de compilation fondée sur des contraintes différées : les sous-termes problématiques sont remplacés par des liens enregistrant des contraintes qui seront vérifiées au cours de l'exécution, ce qui permet de retrouver le comportement d'unification attendu.

Nous étudions également la prédictibilité des programmes logiques au moyen d'une analyse de déterminisme pour ELPI. Celle-ci garantit qu'un prédicat donné produit au plus une solution, en supportant prédicats d'ordre supérieur, clauses dynamiques et cut. Cette analyse est particulièrement pertinente dans le contexte de la résolution de classes de types, où un comportement déterministe est généralement attendu. Enfin, nous formalisons la correction de cette analyse dans Rocq via un modèle d'ordre un de l'interpréteur ELPI et une preuve de sûreté du vérificateur statique.

Dans l'ensemble, ce travail met en évidence le rôle des techniques de programmation logique dans la mise en œuvre et l'analyse de mécanismes d'élaboration, éclairant les liens entre résolution de classes de types, programmation logique et vérification formelle.

Mots-clés : Programmation logique, Théorie des types, Elpi, Rocq, Classes de type, Analyse de déterminisme.